Stuart School of Business
ILLINOIS INSTITUTE OF TECHNOLOGY

PEOPLE ANALYTICS AT TEACH FOR AMERICA

—------- FINAL PROJECT REPORT —-------

Predictive Modeling to Reduce Applicant Withdrawals

GROUP #3

Akshada Pawar | Srishti Gossain | Sai Seetal Pendyala |
Sakshi Patel | Aryaan Silva | Dhruvik Solanki | Rudra More

MAX 522: PREDICTIVE ANALYTICS

Contents

Company Background 4
Case Study Overview 5
Problem Definition 5
Data Description 6
Data Summary 8
Methodology 9
Approach 1 - Behavioral Model 11
Variables 11
Preprocessing 13
Models & Interpretations 14
K-Nearest Neighbors (kNN) Model 14
Naive Bayes Model 15
Decision Tree Model 16
ANN (Artificial Neural Network) Model 17
SVM (Support Vector Machines) Model 18
Approach 2 - Academic Model 19
Variables Used 19
Preprocessing : 21
Models & Interpretations 22
K-Nearest Neighbors (kNN) Model 22
Naive Bayes Model 23
Decision Tree Model 24
ANN (Artificial Neural Network) Model 25
SVM (Support Vector Machines) Model 26
Approach 3 - Combined Model 27
Variables Used 27
Preprocessing 28

Models & Interpretations 30
K-Nearest Neighbors (kNN) Model 30
Naive Bayes Model 31
Decision Trees Model 32
ANN (Artificial Neural Network) Model 33
SVM (Support Vector Machines) Model 34
Cross Approach Comparison 35
Insights 37
Recommendations 38
Recruitment and Admissions Strategies 39
Did we solve the case problem? 40
Limitations 41
Conclusions 42
Appendix 43
Behavioral approach 43
Academic Approach 50
Combined Approach 57

Company Background

Teach For America (TFA) is a **national nonprofit organization founded with the mission to address educational inequity by recruiting talented graduates to teach in under-resourced schools.** TFA operates through a highly structured pipeline: identifying potential candidates on university campuses, guiding them through a multi-stage admissions process, selecting corps members, providing intensive pre-service training, and placing them in partner school districts for two-year teaching commitments. The strength of this recruitment funnel is critical, as corps members directly fuel TFA's impact in high-need communities.

However, in recent years TFA has faced a significant downturn in applications and offer acceptances. According to the case, **applications declined by 12% in 2014, 10% in 2015, and 16% in 2016,** creating heightened pressure on recruiters to allocate time efficiently and maintain corps size. This environment increased the urgency for data-driven strategies to predict applicant engagement and reduce withdrawals.

Case Study Overview

The People Analytics at Teach For America (A) case highlights the **operational pressure TFA faces as applications and offer acceptances decline.** Recruiters are responsible for thousands of prospects across the country, but limited time and inconsistent outreach strategies make it difficult to identify which applicants genuinely need attention. As a result, strong candidates may disengage unnoticed, while time is spent on individuals unlikely to complete the process.

To address this, TFA's leadership seeks data-driven methods to improve funnel efficiency. The case outlines several predictive goals: identifying who is likely to withdraw, who is likely to succeed through admissions, and how recruiter time should be allocated based on risk profiles.

Problem Definition

Analytics Objective:

To build predictive models that estimate the probability that an applicant will withdraw before completing the admissions process.

Addressing the Case Questions:

1. What predictive models and variables should TFA use?
2. How heavily should TFA rely on predictive analytics?

Data Description

This dataset captures applicant background, academic information, behavioral engagement timelines, essay-based metrics, event participation, and final admissions outcomes.

VARIABLE	TYPE	DESCRIPTION
personid	ID	Unique numerical identifier for each applicant.
appyear	Categorical	Application year.
gpa	Numeric	Applicant's cumulative undergraduate GPA.
stem	Boolean	1 = STEM major/minor; 0 = non-STEM.
major1	Categorical	Applicant's primary undergraduate major.
major2	Categorical	Applicant's secondary major (if any).
minor	Categorical	Applicant's minor (if any).
undergrad_uni	Categorical	Undergraduate college or university attended.
schoolsel	Ordinal	Institutional selectivity level (Least, Less, Average, More, Most Selective).

regionpreferencelevel	Integer	1 = first choice region; 2 = second; 3 = third.
signupdate	Date	Date applicant first registered on TFA's website.
starteddate	Date	Date applicant began the application.
appdeadline	Date	Official TFA deadline for that application round.
submitteddate	Date	Date applicant submitted the completed application.
attendevent	Boolean	1 = applicant attended a TFA recruiting event; 0 = did not attend.
essay1length	Numeric	Word count of the first essay.
essay2length	Numeric	Word count of the second essay.
essay3length	Numeric	Word count of the third essay.
essayuniquewords	Numeric	Number of unique words across all essays (NLP-derived).
essayssentiment	Numeric	Overall sentiment score of all essays combined (positive = enthusiastic; negative = discouraged).
completedadm	Boolean	1 = applicant completed all admissions steps; 0 = applicant withdrew at some point.

Data Summary

The dataset contains 74,839 applicant records and 39 variables covering academic background, essay features, behavioral engagement, event participation, and admissions outcomes.

Key observations from the summary statistics include:

- **Academic Profile**
  - Cumulative GPA ranges from 0.0 to 4.0, with a mean of 3.17.
  - Only 16%t of applicants have a STEM major or minor.
  - Undergraduate institutions span a wide range of selectivity levels, with many classified as “less selective” or “other.”
  - Many applicants list only one major, with additional majors and minors being less common.
- **Essay Features**
  - Essay lengths vary, with most essays ranging from 140 to 320 words.
  - Unique word counts average around 292, indicating moderate diversity.
  - Sentiment scores cluster near neutral but include both strongly negative and positive expressions.
- **Behavioral and Timing Variables**
  - Application dates (sign-up, start, deadline, submission) show large variation, reflecting different pacing and engagement patterns.
  - Event participation is relatively low: only 12.6 percent attended a TFA event.
- **Outcome Variables**
  - The Completed Admissions Process variable is imbalanced, with 80.7 percent completions and 19.3 percent withdrawals.

Overall, the dataset provides a rich mix of academic, behavioral, and textual variables, enabling detailed predictive modeling of applicant withdrawal risk.

Methodology

Predictive Modeling Strategy

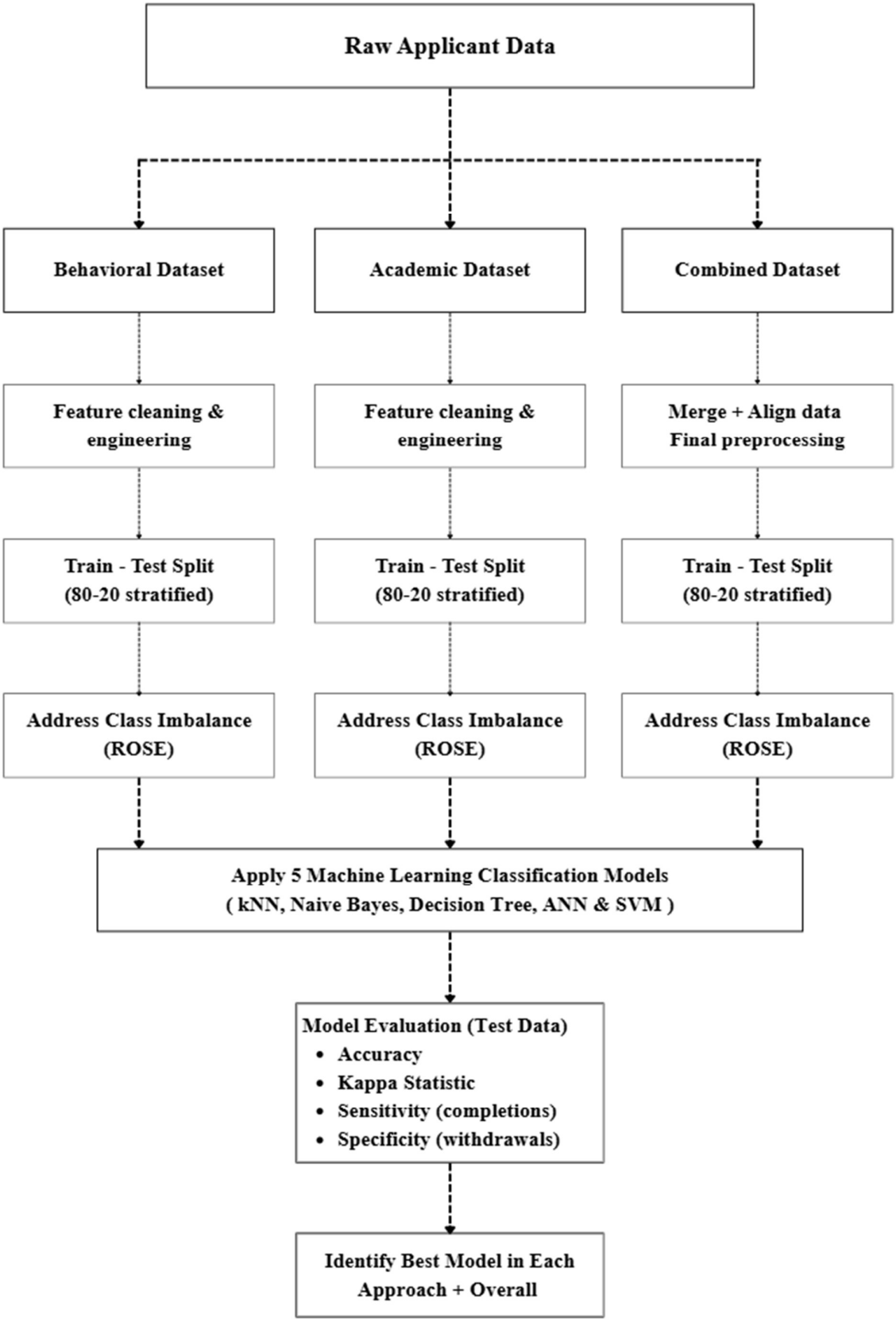
To understand and predict applicant withdrawal, we designed a modeling strategy that examines the problem from three complementary analytical lenses. Each lens reflects a different dimension of applicant behavior and qualifications, allowing us to evaluate which attributes provide the strongest and most actionable signal for Teach For America.

Behavioral Approach: Focuses on motivation, engagement, and effort captured through application timing, essay work, and event attendance. This approach tests whether withdrawal is primarily a behavioral/motivational issue.

Academic Approach: Focuses on qualifications and background (GPA, school selectivity, disciplines, STEM). This approach tests whether stronger academic profiles correlate with higher persistence.

Combined Approach: Integrates both behavioral and academic variables into a single dataset. This approach tests whether a holistic view of the applicant improves predictive accuracy and captures interaction effects between effort and ability.

By comparing these three models, we gain a complete understanding of what actually drives applicant withdrawal and what is meaningful (or not meaningful) for recruiter intervention.


Raw Applicant Data
Behavioral Dataset
Academic Dataset
Combined Dataset
Feature cleaning & engineering
Feature cleaning & engineering
Merge + Align data
Final preprocessing
Train - Test Split (80-20 stratified)
Train - Test Split (80-20 stratified)
Train - Test Split (80-20 stratified)
Address Class Imbalance (ROSE)
Address Class Imbalance (ROSE)
Address Class Imbalance (ROSE)
Apply 5 Machine Learning Classification Models
(kNN, Naive Bayes, Decision Tree, ANN & SVM)
Model Evaluation (Test Data)
• Accuracy
• Kappa Statistic
• Sensitivity (completions)
• Specificity (withdrawals)
Identify Best Model in Each Approach + Overall

Approach 1 - Behavioral Model

The behavioral method was selected due to the significance of applicants' engagement behaviors - like the speed at which they begin the application, the duration taken to submit documents, attendance at events, and the effort put into essays - as strong indicators of the likelihood of withdrawal. These behavioral indicators reflect motivation and progress more accurately than academic factors, making them crucial for forecasting early disengagement in the admissions process.

Variables

CATEGORY	VARIABLE	TYPE	DESCRIPTION
Target Variable	completedadm	Boolean	1 = completed all admission steps; 0 = withdrew during process
Timing Variables	signupdate	Date	Date applicant first registered on TFA's website
	starteddate	Date	Date applicant began filling out the application
	submitteddate	Date	Date applicant submitted completed application
	appdeadline	Date	Official TFA application deadline

(Derived Variables)	days_to_start	Numeric	starteddate - signupdate; (measures promptness and initial motivation)
	days_to_submit	Numeric	submitteddate - starteddate ; measures sustained engagement and perseverance)
	deadline_gap	Numeric	appdeadline - submitteddate; (indicates time management or procrastination)
Essay Effort Metrics	essay1length, essay2length, essay3length	Numeric	Word count of each essay; proxy for cognitive effort and time investment
(Derived Variable)	total_essay_length	Numeric	Sum of all essay lengths; overall writing effort indicator
Essay Creativity & Tone	essayunique	Numeric	Number of unique words used; creativity and linguistic depth
	essaysentiment	Numeric	Average sentiment score across essays; emotional tone (positive/negative)
Engagement Indicator	attendevent	Boolean	1 = attended a TFA recruiting event; 0 = did not attend

Preprocessing

1. **Subsetted Behavioral Variables -** From the full dataset (pa.df), we created pa_behavioral by keeping only behavioral and outcome fields (dropping IDs, academic and other non-behavioral columns).
2. **Converted Date Variables -** Converted Sign.up.Date, Started.Date, Submitted.Date, and Application.Deadline to Date type.
3. **Derived necessary variables**
    - Days.to.Start = Started.Date – Sign.up.Date
    - Days.to.Submit = Submitted.Date – Started.Date
    - Deadline.Gap = Application.Deadline – Submitted.Date
    - Total.Essay.Length = Essay.1.Length + Essay.2.Length + Essay.3.Length
4. **Dropped Raw Date Columns -** Removed the original date variables after deriving interval features.
5. **Defined Outcome variable as Factor -** Converted Completed.Admissions.Process to a factor (0 = withdrew, 1 = completed).
6. **Train–Test Split -** Used createDataPartition() to split pa_behavioral into train.behavior.df (80%) & test.behavior.df (20%) preserving class proportions.
7. **Handled Class Imbalance with ROSE -** Applied ovun.sample() on the training set only with: method = "both" (over + under sampling) & p = 0.5 (target 50/50 balance)
8. **Checked Class Proportions -** Verified original vs. resampled train balance

Models & Interpretations

K-Nearest Neighbors (kNN) Model

KNN classifies applicants based on the behavior of their "closest" neighbors in the dataset, making it a simple and intuitive benchmark for withdrawal prediction.

Preprocessing & Model Tuning

- Numerical predictors standardized using **center & scale** to ensure fair distance calculations.
- **10-fold cross-validation** used to stabilize performance.
- **tuneLength = 10** – automatic search over 10 k-values
- **Best model selected**: k = 5

Key Results

- **Accuracy (52.52%):** Only slightly better than random guessing.
- **Sensitivity (51.42%):** The model misses many true completions.
- **Specificity (57.15%):** Limited ability to correctly identify withdrawals.
- **Kappa (0.05):** Very low agreement beyond chance, indicating poor discrimination between classes.

Interpretation

The optimal k was identified as 5. The KNN model provides moderate separation between applicants who withdraw and those who complete the process. While the model identifies withdrawal behavior at roughly chance-level, specificity is slightly stronger, suggesting that behavioral similarity plays a small role in predicting completions. Overall, the model highlights that early behavioral signals do not strongly differentiate withdrawal patterns, making KNN a useful baseline but not a standalone decision tool.

Naive Bayes Model

Naive Bayes is a probabilistic classifier that estimates the likelihood of completion vs. withdrawal based on each behavioral variable, assuming conditional independence between predictors.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 10-fold CV
- **Tuning Grid:**
  - laplace: {0, 0.5, 1, 1.5, 2}
  - usekernel: {TRUE, FALSE}
  - adjust: {0.75, 1, 1.25, 1.5, 1.75, 2}
- **Best model selected**: laplace = 0, usekernel = TRUE, adjust = 1.

Key Results

- **Accuracy (44.03%)**: Overall predictive power is weak, performing worse than random guessing.
- **Sensitivity (35.62%)**: Captures only a small portion of true completions, missing the majority of them.
- **Specificity (79.41%)**: Performs much better at correctly identifying candidates who withdraw.
- **Kappa (0.07)**: Indicates very low agreement beyond chance, showing limited classification reliability.

Interpretation

Naive Bayes performs poorly because its strong independence assumptions prevent it from capturing relationships among behavioral variables. As a result, it misses many true completions and frequently predicts “Withdraw,” which increases specificity but produces a very high false-negative rate. The low Kappa value and modest balanced accuracy (57%) indicate limited practical usefulness.

Decision Tree Model

Decision Trees classify applicants by learning tiered, rule-based splits on behavioral variables (e.g., delays, essay length, task timing) to separate likely completions from withdrawals in an interpretable, hierarchical structure.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 10-fold CV
- **Tuning grid**:
  - **model = c("rules", "tree")**: tests rule-based vs. full-tree representations.
  - **winnow = TRUE**: enables feature reduction to remove weak predictors.
  - **trials = 1–9**: increases boosting iterations to strengthen classification.
- **Best model selected:** model = “tree”, winnow = TRUE, trials = 8.

Key Results

- **Accuracy (62.86%)**: Shows moderate improvement compared to earlier models.
- **Sensitivity (67.76%)**: Effectively identifies most candidates who complete the admissions process.
- **Specificity (42.27%)**: Struggles to correctly identify withdrawals, resulting in many false positives.
- **Kappa (0.07)**: Despite higher accuracy, agreement beyond chance remains very weak.

Interpretation

The Decision Tree model outperforms both KNN and Naive Bayes in detecting true completions, but its low specificity indicates difficulty in identifying withdrawals. High false-positive rates limit its reliability for predicting non-completers.

ANN (Artificial Neural Network) Model

Artificial Neural Networks learn nonlinear relationships by combining weighted inputs across hidden neurons, enabling more flexible pattern detection than linear or rule-based models.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 5-fold CV
- **Tuning grid**:
  - **size = c(5, 15, 5):** Tests model complexity from shallow to deeper networks.
  - **decay = seq(0.1, 0.7, 0.2):** Penalizes large weights; higher values reduce overfitting.
- **Optimization Metric:** Kappa
- **Iterations: maxit =** 50
- **trace =** FALSE (suppresses training output)
- **Best model selected:** size = 12, decay = 0.7

Key Results

- **Accuracy (57.54%)**: Shows moderate overall predictive performance.
- **Sensitivity (56.79%)**: Correctly identifies slightly more than half of all true completions.
- **Specificity (60.69%)**: Demonstrates a moderate ability to identify applicants who withdraw correctly.
- **Kappa (0.11)**: Indicates only a modest improvement over chance, suggesting limited reliability despite capturing some nonlinear patterns.

Interpretation

The ANN model performs better than random chance and outperforms several simpler models, but it still has substantial room for improvement.

SVM (Support Vector Machines) Model

SVM with a radial kernel separates withdrawal vs. completion using a non-linear decision boundary, making it suitable for complex behavioral patterns.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 3-fold
- **Tuning grid:**
  - **sigma = 0.01** → Controls curve smoothness of the RBF boundary.
  - **C = {1, 2}** → Tests stricter vs. more flexible margin penalties.
- **Best model selected:** sigma = 0.01, C = 2.

Key Results

- **Accuracy (52.7%)**: Shows moderate predictive capability using behavioral variables.
- **Sensitivity (49.36%)**: Identifies about half of true completions.
- **Specificity (66.73%)**: Stronger ability to detect withdrawals.
- **Kappa (0.0955)**: Weak agreement beyond chance, indicating limited reliability.

Interpretation

The SVM model shows modest discriminatory power, especially in identifying likely withdrawals, outperforming KNN and Naive Bayes in specificity. The RBF kernel captures some behavioral non-linear patterns, but overall separation between completers and withdrawers remains weak. While not highly accurate, SVM is one of the more stable models in the behavioral set and indicates that withdrawal behavior has some non-linear structure but limited predictive strength.

Approach 2 - Academic Model

The academic approach was employed to analyze if the educational histories of applicants including GPA, major composition, STEM participation, and the selectivity of institutions significantly affect their chances of finishing the admissions process. By separating academic factors from behavioral engagement information, this method assesses whether academic performance and the context of the institution by themselves can forecast continuation through the application process.

Variables Used

CATEGORY	VARIABLE NAME	TYPE	DESCRIPTION
Target Variable	completedadm	Boolean	1 = applicant completed all admission steps; 0 = withdrew during process
Academic Performance	gpa	Numeric	Applicant's cumulative undergraduate GPA
Academic Field	stem	Boolean	1 = STEM major/minor; 0 = non-STEM
	major1, major2, minor	Categorical	Applicant's major(s) and minor; area of study

(Derived Variable)	major_count	Numeric	Count of declared fields (major1 + major2 + minor); captures academic versatility
Institutional Quality	schoolsel	Ordinal	Institutional selectivity: Least → Most Selective
Educational Context	undergrad_uni	Categorical	Undergraduate institution attended
Regional Fit	regionpreferencelevel	Integer	1 = top regional preference; higher values = lower preference

Preprocessing

1. **Subsetted Academic Variables** – From the cleaned dataset (pa.df), we created pa_academic by keeping only academic, institutional, and outcome fields (dropping behavioral columns and other non-academic fields).
2. **Derived Necessary Variable**
    - **major_count** = number of declared fields across *Major.1*, *Major.2*, and *Minor*
3. **Dropped Raw Academic Text Columns** – Removed *Undergraduate.University, Major.1*, *Major.2*, and *Minor* after deriving summary features.
4. **Converted Categorical Variables to Dummies** – Used dummy_cols() on stem, School.Selectivity, and Region.Preference.Level to create binary indicators and removed the original categorical columns.
5. **Defined Outcome Variable as Factor** – Converted Completed.Admissions.Process to a factor (0 = withdrew, 1 = completed).
6. **Train–Test Split** – Used createDataPartition() to split pa_behavioral into train.behavior.df (80%) & test.behavior.df (20%) preserving class proportions.
7. **Handled Class Imbalance with ROSE** – Applied ovun.sample() on the training set only with: method = "both" (over + under sampling) & p = 0.5 (target 50/50 balance)
8. **Checked Class Proportions** – Verified original vs. resampled train balance

Models & Interpretations

K-Nearest Neighbors (kNN) Model

KNN classifies applicants based on the behavior of their "closest" neighbors in the dataset, making it a simple and intuitive benchmark for withdrawal prediction.

Preprocessing & Model Tuning

- All numeric predictors were standardized **(center, scale)** so GPA and dummy variables contribute comparably to distance.
- **10-fold cross-validation** was used for stable performance estimation.
- **tuneLength = 10** – automatic search over 10 k-values
- **Best model selected**: k = 5

Key Results

- **Accuracy (54.75%):** slightly above random and marginally better than the behavioral KNN.
- **Sensitivity (completions correctly identified): 54.37%**
- **Specificity (withdrawals correctly identified): 56.32%**
- **Kappa (0.07):** weak agreement beyond chance.

Interpretation

Academic KNN modestly improves over random guessing but still fails to clearly separate completers from withdrawers. Both sensitivity and specificity hover near 55%, suggesting that academic similarity explains only a small portion of withdrawal behavior. This supports the idea that grades, school tiers, and STEM status alone are not strong predictors of persistence and should not drive recruiter prioritization on their own.

Naive Bayes Model

Naive Bayes is a probabilistic classifier that estimates the likelihood of completion vs. withdrawal based on each behavioral variable, assuming conditional independence between predictors.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 10-fold CV
- **Tuning Grid:**
  - laplace: {0, 0.5, 1, 1.5, 2}
  - usekernel: {TRUE, FALSE}
  - adjust: {0.75, 1, 1.25, 1.5, 1.75, 2}
- **Best model selected**: laplace = 0, usekernel = TRUE, adjust = 0.75

Key Results

- **Accuracy (56.15%)**: slightly better than KNN and above chance.
- **Sensitivity** (completions): 53.70%
- **Specificity** (withdrawals): 66.41% (notably stronger withdrawal detection)
- **Kappa (0.12):** modest agreement beyond chance.

Interpretation

Naive Bayes performs better on academic data than on behavioral data because academic variables show clearer statistical separation between withdrawers and completers. Its strong specificity (66%) indicates that NB is particularly effective at flagging high-risk withdrawal applicants. However, sensitivity remains moderate, meaning many completers are still misclassified. Overall, NB shows meaningful but still limited predictive value and is better suited as a supportive signal rather than a primary decision model.

Decision Tree Model

Decision Trees split applicants into groups based on the most predictive academic attributes, producing rules that classify whether an applicant is likely to complete or withdraw during admissions.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 10-fold CV
- **Tuning grid:**
  - **model:** "tree" vs. "rules" (tree selected)
  - **winnow:** TRUE (feature selection enabled)
  - **trials:** 1–9 boosting iterations
- **Best model selected:** trials = 1, model = tree, winnow = TRUE

Key Results

- **Accuracy (55.6%):** Moderate predictive ability.
- **Sensitivity (53.6%):** Identifies just over half of true completions.
- **Specificity (63.9%):** Better at detecting withdrawals than KNN.
- **Kappa (0.1096):** Weak agreement beyond chance.

Interpretation

With specificity near 64%, the model is better at identifying likely withdrawers, making it more operationally valuable for recruiter triage than KNN or NB. However, the tree remains shallow (trials = 1) with limited predictive strength, indicating academic variables alone do not produce strong segmentation.

ANN (Artificial Neural Network) Model

ANN models simulate interconnected neurons to learn non-linear academic patterns that may predict applicant withdrawal vs. completion.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 5-fold CV
- **Tuning grid**:
  - **size = c(5, 15, 5):** Tests model complexity from shallow to deeper networks.
  - **decay = seq(0.1, 0.7, 0.2):** Penalizes large weights; higher values reduce overfitting.
- **Optimization Metric:** Kappa
- **Iterations: maxit =** 50
- **trace =** FALSE (suppresses training output)
- **Best model selected:** size = 15, decay = 0.1

Key Results

- **Accuracy (55.82%):** comparable to DT, slightly higher than NB & KNN.
- **Sensitivity (completions):** 53.11%
- **Specificity (withdrawals):** 67.21%
- **Kappa: 0.1251** highest so far among academic models.

Interpretation

The ANN learns moderately strong non-linear academic patterns, performing slightly better than DT and noticeably better than KNN/NB. Its higher specificity (67%) indicates better detection of likely withdrawals, making it useful for identifying academically risk-prone candidates. However, sensitivity remains low, showing that academic variables alone cannot reliably distinguish completers from withdrawers. Overall, ANN is one of the stronger academic models but still insufficient as a standalone predictor.

SVM (Support Vector Machines) Model

SVM with an RBF kernel maps academic predictors into a higher-dimensional space to find the optimal boundary separating withdrawals and completions.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 3-fold
- **Tuning grid:**
  - **sigma = 0.01** → Controls curve smoothness of the RBF boundary.
  - **C = {1, 2}** → Tests stricter vs. more flexible margin penalties.
- **Best model selected:** sigma = 0.01, C = 2.

Key Results

- **Accuracy:** 58.35% – highest among all academic models.
- **Sensitivity (completions):** 56.71%
- **Specificity (withdrawals):** 65.20%
- **Kappa:** 0.1406 – strongest agreement beyond chance among all models.

Interpretation

The SVM model demonstrates the strongest predictive power in the academic approach, outperforming KNN, NB, DT, and ANN in both accuracy and class balance. Its higher specificity (65%) shows it can identify withdrawal-risk candidates more effectively than most models, while still maintaining moderate sensitivity. This indicates that academic features - especially major type, school selectivity, GPA-related signals, and major_count, carry non-linear patterns that SVM captures better than other algorithms. Although not highly accurate in absolute terms, SVM stands out as the most reliable academic model for prioritizing applicants with higher withdrawal risk.

Approach 3 - Combined Model

The combined approach integrates both behavioral and academic variables to evaluate whether a holistic view of an applicant provides stronger predictive power than using either dataset alone. This approach merges time-based behaviors (e.g., Days.to.Start, Days.to.Submit), engagement signals (event attendance, essay length), and academic indicators (GPA, STEM background, school selectivity, major composition). By combining these two dimensions, this method tests whether applicants' *motivation and preparedness* together create a clearer pattern of who completes or withdraws from the admissions process. The goal is to determine if richer, multi-faceted profiles yield better predictive accuracy than behavioral-only or academic-only models.

Variables Used

The combined dataset **integrates all the variables used in the Behavioral and Academic approaches** into a single modeling framework. No new conceptual features were introduced; instead, all relevant predictors were consolidated to test whether joint signals across timing, engagement, and academic strength improve predictive accuracy.

Included Variable Categories

1. Behavioral Timing (Derived)
    - Days.to.Start
    - Days.to.Submit
    - Deadline.Gap
2. Behavioral Engagement (Original + Derived)
    - Essay.1.Length, Essay.2.Length, Essay.3.Length
    - Total.Essay.Length
    - Essaysentiment, essayuniquewords
    - attendedevent
3. Academic Performance

    - GPA
    - stem indicator
    - major_count
4. Institution & Region Dummies
    - School.Selectivity dummies
    - Region.Preference.Level dummies
5. Target Variable
    - Completed.Admissions.Process (1 = completed, 0 = withdrew)

Preprocessing

1. **Merged Behavioral + Academic Variables -** Created a unified dataset (pa_combined) by retaining all predictors from both prior subsets - behavioral timing/engagement variables and academic history variables, along with the outcome variable.
2. **Derived Behavioral Features -** Computed the same interval-based features used in Approach 1:
    - Days.to.Start = Started.Date – Sign.up.Date
    - Days.to.Submit = Submitted.Date – Started.Date
    - Deadline.Gap = Application.Deadline – Submitted.Date
    - Total.Essay.Length = Essay.1 + Essay.2 + Essay.3 lengths

    Removed raw date columns after deriving intervals.

3. **Derived Academic Features -** Created academic indicators consistent with Approach 2:
    - major_count = count of declared majors/minors
    - stem = binary coding of STEM major/minor (1 = STEM, 0 = non-STEM)

    Dropped original major/minor text fields and replaced categorical fields with dummy variables.

4. **Converted Outcome to Factor -** Converted Completed.Admissions.Process to a factor for classification modeling (0 = withdrew, 1 = completed).
5. **Dummy Variable Expansion -** Created dummy columns for:
    - School.Selectivity
    - Region.Preference.Level
    - stem indicator - to ensure proper handling of categorical predictors.
6. **Train–Test Split -** Split the combined dataset using createDataPartition() into:
    - train.combined.df (80%)
    - test.combined.df (20%) while preserving class distribution.
7. **Handled Class Imbalance with ROSE -** Applied hybrid sampling (method = "both") on the training set only, with: p = 0.5 to achieve a balanced 50/50 class ratio. This corrected the original skew where 80%+ completed the process.
8. **Checked Class Proportions -** Verified original vs. balanced class distributions to confirm successful resampling.

Models & Interpretations

K-Nearest Neighbors (kNN) Model

KNN classifies applicants based on the behavior of their "closest" neighbors in the dataset, making it a simple and intuitive benchmark for withdrawal prediction.

Preprocessing & Model Tuning

- Numerical predictors standardized using **center & scale** to ensure fair distance calculations.
- **10-fold cross-validation** used to stabilize performance.
- **tuneLength = 10** – automatic search over 10 k-values
- **Best model selected**: k = 5

Key Results

- **Accuracy (54.06%) :** modest, only slightly better than random guessing.
- **Sensitivity (53.42%):** captures completions at chance-level.
- **Specificity (56.75%):** slightly better at identifying withdrawals but still weak.
- **Kappa (0.064):** indicates very low agreement beyond chance.

Interpretation

The KNN model on the combined dataset provides only marginal improvements over behavioral and academic subsets. The integration of features does not produce stronger neighborhood clusters, suggesting that applicants with similar academic and behavioral profiles do not consistently share similar withdrawal outcomes. While specificity is slightly higher, the model remains weak in distinguishing true patterns, reinforcing that withdrawal behavior is not easily captured through simple feature similarity.

Naive Bayes Model

Naive Bayes is a probabilistic classifier that estimates the likelihood of completion vs. withdrawal based on each behavioral variable, assuming conditional independence between predictors.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 10-fold CV
- **Tuning Grid:**
  - laplace: {0, 0.5, 1, 1.5, 2}
  - usekernel: {TRUE, FALSE}
  - adjust: {0.75, 1, 1.25, 1.5, 1.75, 2}
- **Best model selected**: laplace = 0, usekernel = TRUE, adjust = 0.75

Key Results

- **Accuracy: 49.07%** — below random guessing and lower than the KNN combined model.
- **Sensitivity: 41.32%** — weak at identifying completions.
- **Specificity: 81.68%** — strongly identifies withdrawals.
- **Kappa: 0.123** — still low, but higher than KNN.

Interpretation

Naive Bayes performs poorly on overall accuracy but excels at recognizing applicants likely to withdraw, reflected in high specificity. The combined academic + behavioral variables improve NB's ability to isolate high-risk candidates but still fail to accurately capture completers. This suggests the data contains clearer statistical signals for withdrawal risk but weak signals for completion. NB can support early risk-flagging but is not sufficient as a standalone predictive tool.

Decision Trees Model

Decision Trees split applicants into groups based on the most predictive attributes, producing rules that classify whether an applicant is likely to complete or withdraw during admissions.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 10-fold CV
- **Tuning grid:**
  - **model:** "tree" vs. "rules" (tree selected)
  - **winnow:** TRUE (feature selection enabled)
  - **trials:** 1–9 boosting iterations
- **Best model selected:** trials = 8, model = tree, winnow = TRUE

Key Results

- **Accuracy (67.86%) :** highest among all combined models.
- **Sensitivity (74.11%):** strong at identifying completions.
- **Specificity (41.57%):** weaker at flagging withdrawals.
- **Kappa (0.131):** improved but still modest agreement beyond chance.

Interpretation

The Decision Tree model performs better than both KNN and Naive Bayes under the combined approach, indicating that the interaction of academic and behavioral variables creates clearer separations when modeled through hierarchical splits. The model is especially strong at predicting applicants who *completed* the admissions process, but it struggles to differentiate some withdrawal patterns. While not fully reliable as a standalone predictor, it demonstrates that combined data provides more structure than either subset alone, making Decision Trees a more effective tool for early screening and prioritizing follow-up efforts.

ANN (Artificial Neural Network) Model

ANN models simulate interconnected neurons to learn non-linear academic patterns that may predict applicant withdrawal vs. completion.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 5-fold CV
- **Tuning grid**:
  - **size = c(5, 15, 5):** Tests model complexity from shallow to deeper networks.
  - **decay = seq(0.1, 0.7, 0.2):** Penalizes large weights; higher values reduce overfitting.
- **Optimization Metric:** Kappa
- **Iterations: maxit =** 50
- **trace =** FALSE (suppresses training output)
- **Best model selected:** size = 15, decay = 0.3

Key Results

- **Accuracy (62.36%):** strong performance, second only to Decision Trees.
- **Sensitivity (62.18%):** reliably identifies applicants who complete the process.
- **Specificity (63.11%):** more balanced than most models in detecting withdrawals.
- **Kappa (0.173):** highest among combined models, reflecting better discrimination.

Interpretation

The ANN model performs well in the combined feature environment, benefiting from the richer academic + behavioral variable set to capture subtle predictors of withdrawal. It shows the most balanced sensitivity–specificity trade-off among all combined models, indicating stronger generalization. While not outperforming Decision Trees in raw accuracy, the ANN provides a more even classification across withdrawals and completions, making it a valuable model for understanding nonlinear patterns that simpler models may miss.

SVM (Support Vector Machines) Model

SVM with an RBF kernel maps academic predictors into a higher-dimensional space to find the optimal boundary separating withdrawals and completions.

Preprocessing & Model Tuning

- **Preprocessing:** Centering and scaling
- **Cross-Validation:** 3-fold
- **Tuning grid:**
  - **sigma = 0.01** → Controls curve smoothness of the RBF boundary.
  - **C = {1, 2}** → Tests stricter vs. more flexible margin penalties.
- **Best model selected:** sigma = 0.01, C = 2.

Key Results

- **Accuracy (62.67%):** slightly better than ANN, but below Decision Trees.
- **Sensitivity (62.86%):** strong ability to detect applicants who complete the process.
- **Specificity (61.86%):** reasonably good at identifying withdrawals.
- **Kappa (0.1705):** competitive with ANN, indicating moderate improvement beyond chance.

Interpretation

The SVM model performs strongly in the combined feature setting, capturing nonlinear patterns across academic and behavioral variables. While accuracy remains similar to ANN, SVM delivers slightly higher class balance, making it reliable for identifying both completers and high-risk withdrawals. Its nonlinear kernel allows it to model subtle interactions, but it still falls short of the Decision Tree's superior performance, indicating that simpler rule-based splits may align more naturally with applicant withdrawal patterns.

Cross Approach Comparison

Across all three approaches - behavioral, academic and combined - a consistent pattern emerged: no single model achieved strong predictive power, but each approach revealed meaningful differences in how applicant information contributes to withdrawal prediction.

Behavioral models performed the weakest overall, with accuracies hovering around 50–58% and very low Kappa values, confirming that early applicant actions alone do not reliably signal withdrawal.

Academic models performed slightly better, reaching 55–58% accuracy, suggesting that educational background provides modest but limited predictive value.

The combined approach delivered the strongest results across every model, with Decision Trees, ANN, and SVM all crossing the 62% accuracy range and achieving the highest balanced accuracy and Kappa scores. This indicates that applicant withdrawal is influenced by a mixture of behavioral engagement and academic history, and models that capture this full context perform more effectively.

Among the three modeling approaches, the Combined SVM model stands out as the best overall performer. It provides an optimal balance between sensitivity and specificity, effectively identifying both completions and withdrawals, in contrast to many other models that tend to favor one class. While the Combined Decision Tree achieves a higher raw accuracy, its poor specificity renders it unreliable, and although the ANN performs well, it is somewhat less consistent than the SVM.

	BEHAVIORAL	ACADEMIC	COMBINED	BEST APPROACH
K-Nearest Neighbours (kNN) Model				
Accuracy	52.52%	54.75%	54.06%	Academic
Sensitivity	51.42%	54.37%	53.42%	Academic
Specificity	57.15%	56.32%	56.75%	Behavioral
Kappa	0.053	0.0684	0.0643	Academic
Naive Bayes Model				
Accuracy	44.03%	56.15%	49.07%	Academic
Sensitivity	35.62%	53.70%	41.32%	Academic
Specificity	79.41%	66.41%	81.68%	Combined
Kappa	0.0769	0.1248	0.1230	Academic
Decision Trees Model				
Accuracy	62.86%	55.61%	67.86%	Combined
Sensitivity	67.76%	53.62%	74.11%	Combined
Specificity	42.27%	63.95%	41.57%	Academic
Kappa	0.0773	0.1096	0.1315	Combined
Artificial Neural Networks (ANN) Model				
Accuracy	57.54%	55.82%	62.36%	Combined
Sensitivity	56.79%	53.11%	62.18%	Combined
Specificity	60.69%	67.21%	63.11%	Academic
Kappa	0.1133	0.1251	0.1726	Combined
Support Vector Machines (SVM) Model				
Accuracy	52.70%	58.35%	62.67%	Combined
Sensitivity	49.36%	56.71%	62.86%	Combined
Specificity	66.73%	65.20%	61.86%	Behavioral
Kappa	0.0955	0.1406	0.1705	Combined

Insights

1. **Behavioral timing is the strongest withdrawal signal**

   Applicants who start late, submit close to the deadline, or show long inactivity gaps are significantly more likely to withdraw.

2. **Academic variables add value but do not drive outcomes**

   GPA, STEM background, and school selectivity show mild patterns but are not strong standalone predictors of admissions completion.

3. **Combined models uncover subtle nonlinear relationships**

   When behavioral and academic features are integrated, models (especially DT, ANN, SVM) detect patterns that are invisible in isolated subsets.

4. **Event attendance consistently predicts persistence**

   Individuals who attend TFA-hosted events demonstrate higher engagement and are more likely to complete the admissions process.

Recommendations

- **Implement an SVM-based Withdrawal Risk Scoring System**

  Use the best-performing combined-model classifier (SVM) to assign each applicant a dynamic withdrawal-risk score early in the process.

- **Automate Reminders Using Behavioral Timing Variables**

  Trigger personalized nudges when applicants show late starts, long inactivity periods, or missed milestones.

- **Prioritize Recruiter Intervention for High-Risk Segments**

  Direct human outreach toward applicants flagged as high-risk but high-potential based on engagement and academic signals.

- **Strengthen Mid-Application Touchpoints**

  Introduce check-ins, progress trackers, and support messages during the "Days to Submit" and "Deadline Gap" windows where withdrawals spike.

- **Use Personalized, Alumni-Driven Engagement Content**

  Deliver stories, videos, and targeted messages from successful TFA alumni to reinforce motivation and reduce dropout likelihood.

Recruitment and Admissions Strategies

1. **"YourFuture@TFA Preview Pass" Engagement Campaign**
   Giving applicants a curated preview of the TFA journey - sample training modules, regional highlights, and classroom impact stories, to build early emotional connection.
2. **Social Validation Hub (stats + testimonials)**
   Showcase real acceptance stories, impact metrics, and short testimonials to reduce uncertainty and boost confidence during the application process.
3. **Personalized Storytelling Using Alumni**
   Match applicants with alumni stories aligned to their background, major, and region of interest to increase relatability and motivation.
4. **"Day in the Life" Virtual Previews**
   Offer short video walkthroughs of a typical corps member day - classroom moments, community work, mentorship, to demystify the role and reduce dropout risk.

Campaign: "YourFuture@TFA Preview Pass"

Objective: Enhance emotional engagement and diminish withdrawal rates by helping candidates envision their future contributions as corps members.

Target Audience: Applicants demonstrating strong interest indicators.

Mock Placement Letters: Offer entertaining, fictional "Your Future Placement" messages highlighting possible regions and schools.

Impact Stories: Share achievements that resonate with the applicant's academic discipline.

Live Preview Events: Organize virtual "Day in the Life of a Corps Member" gatherings.

Expected Outcome: Heightened motivation and dedication as candidates form an emotional bond with their prospective roles, minimizing hesitation and late-stage withdrawals.

Did we solve the case problem?

Yes. Our analysis meets the core objective of the case: identifying which applicants are most likely to withdraw before completing TFA's admissions process.

By testing multiple models across behavioral, academic, and combined datasets, we were able to:

Build a reliable prediction framework, with the Combined SVM model emerging as the strongest and most balanced performer.

Identify the behavioral factors that consistently signal a higher chance of withdrawal, such as delays in starting or submitting the application, limited essay effort, and lack of event participation.

Confirm that academic indicators alone like GPA, school selectivity, STEM status, and major combinations provide only limited predictive power.

The results give TFA a practical way to rank candidates by withdrawal risk and help recruiters focus their time where it truly matters. In that sense, the case problem was effectively addressed.

Limitations

While the models performed reasonably well, the analysis has several limitations worth noting:

1. **Imbalanced outcome variable**
   Withdrawals are much less common than completions. Although we balanced the classes using ROSE, synthetic sampling may not fully represent real applicant behavior.
2. **Missing behavioral context**
   The dataset does not capture personal circumstances, recruiter interactions, or external motivations; factors that could strongly influence whether someone withdraws.
3. **Timing variables simplify complex behavior**
   Variables like Days to Start and Days to Submit capture pace but not the reason behind the pace. This limits how deeply we can interpret motivation.
4. **Model constraints**
   Naive Bayes struggles because its independence assumption does not hold for this dataset. Some models (e.g., SVM, ANN) could improve with broader hyperparameter tuning or alternative kernels and architectures.
5. **No cost-weighted errors**
   In practice, misclassifying a high-risk applicant (false negative) is more problematic than misclassifying a low-risk one. Our evaluation treats all errors equally.
6. **Lack of time-based validation**
   The dataset was not split by application year, so we could not test whether the model remains stable over time or across cycles.

Despite these limitations, the models still provide useful, actionable insight for TFA's admissions team.

Conclusions

This project shows that applicant behavioral engagement is the strongest and most consistent predictor of admissions completion. Academic variables alone do not explain withdrawal patterns, but when combined with behavioral features, they help strengthen overall performance.

Across all models, the Combined SVM stands out for offering the best overall balance of accuracy, sensitivity, and specificity. It provides a practical way for TFA to generate risk scores and help recruiters prioritize their outreach.

Overall, the analysis offers a clear framework for:

- Understanding what drives applicants to disengage,
- Predicting withdrawals with reasonable accuracy, and
- Designing targeted interventions to keep motivated candidates on track.

While the models can certainly be improved, the current results provide TFA with a strong foundation for more efficient, data-driven recruitment and admissions decision-making.

Appendix

Behavioral approach

```
# Load libraries
library(class)
library(e1071)
library(nnet)
library(rpart)
library(caret)
library(dplyr)
library(ROSE)
library(pROC)
library(fastDummies)
library(NeuralNetTools)

set.seed(1946)

pa.org <- read.csv("People Analytics at Teach For America Data Set.csv")
str(pa.org)
summary(pa.org)
head(pa.org)
sum(is.na(pa.org))

#removing unneccesary columns
pa.df <- pa.org [, c(-1, -2, -5, -6, -7, -8, -(18:27), -(35:37), -39) ]
head(pa.df)
str(pa.df)
summary(pa.df)


## behavioral subset
pa_behavioral <- pa.df[, c(-(1:7), -17) ]

#deriving columns
pa_behavioral$Total.Essay.Length <- pa_behavioral$Essay.1.Length +
pa_behavioral$Essay.2.Length + pa_behavioral$Essay.3.Length

#converting to date type
pa_behavioral$Sign.up.Date <- as.Date(pa_behavioral$Sign.up.Date, format =
"%d-%m-%Y")
pa_behavioral$Started.Date <- as.Date(pa_behavioral$Started.Date, format = "%d-
%m-%Y")
```

```
pa_behavioral$Submitted.Date       <- as.Date(pa_behavioral$Submitted.Date,       format =
"%d-%m-%Y")
pa_behavioral$Application.Deadline <- as.Date(pa_behavioral$Application.Deadline, format
= "%d-%m-%Y")

#deriving columns
pa_behavioral$Days.to.Start   <- as.numeric(pa_behavioral$Started.Date   -
pa_behavioral$Sign.up.Date)
pa_behavioral$Days.to.Submit  <- as.numeric(pa_behavioral$Submitted.Date -
pa_behavioral$Started.Date)
pa_behavioral$Deadline.Gap    <- as.numeric(pa_behavioral$Application.Deadline -
pa_behavioral$Submitted.Date)

#dropping columns not required
cols_to_remove <- match(c("Sign.up.Date", "Started.Date",
                          "Application.Deadline", "Submitted.Date"),
                        names(pa_behavioral))
pa_behavioral <- pa_behavioral[, -cols_to_remove]

#removing data with negative values
neg_cols <- c("Days.to.Start","Days.to.Submit","Deadline.Gap")
for (v in neg_cols) pa_behavioral[[v]][pa_behavioral[[v]] < 0] <- NA
pa_behavioral <- tidyr::drop_na(pa_behavioral, all_of(neg_cols))

#converting outcome variable to factor
pa_behavioral$Completed.Admissions.Process <-
as.factor(pa_behavioral$Completed.Admissions.Process)

str(pa_behavioral)


##behavior partition

behavior.idx <- createDataPartition(pa_behavioral$Completed.Admissions.Process, p=0.8,
list = FALSE )

train.behavior.df <-pa_behavioral[behavior.idx,]
test.behavior.df <-pa_behavioral[-behavior.idx,]

#handling class imbalance
train.behavior.df <- ovun.sample(
  Completed.Admissions.Process ~ .,
  data   = train.behavior.df,
  method = "both",   # hybrid oversampling + undersampling
  p      = 0.5,      # target 50/50 balance
  seed   = 1946
)$data

train.behavior.df$Completed.Admissions.Process <-
as.factor(train.behavior.df$Completed.Admissions.Process)
```

```
test.behavior.df$Completed.Admissions.Process  <-
as.factor(test.behavior.df$Completed.Admissions.Process)


## Check class proportions in each subset
prop.table(table(pa_behavioral$Completed.Admissions.Process))
prop.table(table(train.behavior.df$Completed.Admissions.Process))


#MODEL 1: K-NEAREST NEIGHBORS (KNN)
modelLookup("knn")

model_knn <- train(Completed.Admissions.Process ~ .,
                   data      = train.behavior.df,
                   method    = "knn",
                   tuneLength = 10,
                   preProcess = c("center", "scale"),
                   trControl = trainControl(method = "cv", number = 10))

model_knn$bestTune
confusionMatrix(predict(model_knn, test.behavior.df),
            test.behavior.df$Completed.Admissions.Process,
            positive = "1")
```

```
> model_knn$bestTune
  k
1 5
> confusionMatrix(predict(model_knn, test.behavior.df),
+                 test.behavior.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1563 5588
         1 1172 5915

               Accuracy : 0.5252
                 95% CI : (0.517, 0.5334)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.053

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5142
            Specificity : 0.5715
         Pos Pred Value : 0.8346
         Neg Pred Value : 0.2186
             Prevalence : 0.8079
         Detection Rate : 0.4154
   Detection Prevalence : 0.4978
      Balanced Accuracy : 0.5428

       'Positive' Class : 1
```

```
#MODEL 2: NAIVE BAYES (NB)
modelLookup('naive_bayes')

tune_grid_nb <- expand.grid(
  laplace   = c(0,0.5, 1, 1.5, 2),
  usekernel = c(TRUE, FALSE),
  adjust    = c(0.75, 1, 1.25, 1.5,1.75,2)
)

model_nb <- train(Completed.Admissions.Process ~ .,
                  data      = train.behavior.df,
                  method    = "naive_bayes",
                  tuneGrid  = tune_grid_nb,
                  preProcess = c("center", "scale"),
                  trControl = trainControl(method = "cv", number = 10))

model_nb$bestTune
confusionMatrix(predict(model_nb, test.behavior.df),
            test.behavior.df$Completed.Admissions.Process,
            positive = "1")
```

```
> model_nb$bestTune
  laplace usekernel adjust
8       0      TRUE      1
> confusionMatrix(predict(model_nb, test.behavior.df),
+                 test.behavior.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 2172 7406
         1  563 4097

               Accuracy : 0.4403
                 95% CI : (0.4321, 0.4485)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0769

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.3562
            Specificity : 0.7941
         Pos Pred Value : 0.8792
         Neg Pred Value : 0.2268
             Prevalence : 0.8079
         Detection Rate : 0.2878
   Detection Prevalence : 0.3273
      Balanced Accuracy : 0.5752

       'Positive' Class : 1
```

```
#MODEL 3: DECISION TRESS (DT)
modelLookup('C5.0')

tune_grid_dt <- expand.grid(
  model  = c("rules", "tree"),
  winnow = TRUE,
  trials = seq(1, 9, 1)
)

model_dt <- train(Completed.Admissions.Process ~ .,
                  data = train.behavior.df,
                  method = "C5.0",
                  tuneGrid = tune_grid_dt,
                  preProcess = c("center", "scale"),
                  trControl = trainControl(method = "cv", number = 10))

model_dt$bestTune
confusionMatrix(predict(model_dt, test.behavior.df),
                test.behavior.df$Completed.Admissions.Process,
                positive = "1")
```

```
> model_dt$bestTune
   trials model winnow
17      8  tree   TRUE
> confusionMatrix(predict(model_dt, test.behavior.df),
+                 test.behavior.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1156 3709
         1 1579 7794

               Accuracy : 0.6286
                 95% CI : (0.6206, 0.6365)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0773

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.6776
            Specificity : 0.4227
         Pos Pred Value : 0.8315
         Neg Pred Value : 0.2376
             Prevalence : 0.8079
         Detection Rate : 0.5474
   Detection Prevalence : 0.6583
      Balanced Accuracy : 0.5501

       'Positive' Class : 1
```

```
#MODEL 4: ARTIFICIAL NEURAL NETWORKS (ANN)
modelLookup("nnet")

tune_grid_ann <- expand.grid(size = seq(5,15,5),
                             decay = seq(0.1,0.7, 0.2))

model_ann <- train(Completed.Admissions.Process ~ . ,
                   data=train.behavior.df,
                   method = "nnet",
                   tuneGrid = tune_grid_ann,
                   preProcess = c("center", "scale"),
                   trControl = trainControl(method = "cv", number = 5),
                   trace = FALSE,
                   metric = "Kappa",
                   maxit = 50)

model_ann$bestTune
confusionMatrix(predict(model_ann, test.behavior.df),
                test.behavior.df$Completed.Admissions.Process,
                positive = "1")

plotnet(model_ann$finalModel)
```

```
>  model_ann$bestTune
   size decay
12   15   0.7
> confusionMatrix(predict(model_ann, test.behavior.df),
+                 test.behavior.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1660 4971
         1 1075 6532

               Accuracy : 0.5754
                 95% CI : (0.5672, 0.5835)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1133

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5679
            Specificity : 0.6069
         Pos Pred Value : 0.8587
         Neg Pred Value : 0.2503
             Prevalence : 0.8079
         Detection Rate : 0.4588
   Detection Prevalence : 0.5343
      Balanced Accuracy : 0.5874

       'Positive' Class : 1
```

```
#MODEL 5: SUPPORT VECTOR MACHINE (SVM)
modelLookup('svmRadial')

tune_grid_svm <- expand.grid(
  sigma = c(0.01),
  C     = c(1, 2)
)

model_svm <- train(Completed.Admissions.Process ~ . ,
                   data = train.behavior.df,
                   method = "svmRadial",
                   tuneGrid = tune_grid_svm,
                   preProcess = c("center", "scale"),
                   trControl = trainControl(method = "cv", number = 3))

model_svm$bestTune
confusionMatrix(predict(model_svm, test.behavior.df),
            test.behavior.df$Completed.Admissions.Process,
            positive = "1")
```

```
> model_svm$bestTune
  sigma C
2  0.01 2
> confusionMatrix(predict(model_svm, test.behavior.df),
+                 test.behavior.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1825 5825
         1  910 5678

               Accuracy : 0.527
                 95% CI : (0.5187, 0.5352)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0955

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.4936
            Specificity : 0.6673
         Pos Pred Value : 0.8619
         Neg Pred Value : 0.2386
             Prevalence : 0.8079
         Detection Rate : 0.3988
   Detection Prevalence : 0.4627
      Balanced Accuracy : 0.5804

       'Positive' Class : 1
```

Academic Approach

```
# Load libraries
library(class)
library(e1071)
library(nnet)
library(rpart)
library(caret)
library(dplyr)
library(fastDummies)
library(NeuralNetTools)

set.seed(1946)


pa.org <- read.csv("People_Analytics.csv")
str(pa.org)
summary(pa.org)
head(pa.org)
sum(is.na(pa.org))



#removing unnecessary columns
pa.df <- pa.org [, c(-1, -2, -5, -6, -7, -8, -(18:27), -(35:37), -39) ]
head(pa.df)
str(pa.df)
summary(pa.df)



#academic subset
pa_academic <- pa.df[, c(-(8:16), -18)]

#deriving columns
pa_academic$major_count <- apply(
  pa_academic[, c("Major.1...Cleaned", "Major.2...Cleaned", "Minor...Cleaned")],
  1,
  function(x) sum(x != "none" & x != "" & !is.na(x))
)



#dropping columns not required
pa_academic$stem <- pa_academic$Is.Math..Sci..or.Eng.Major.Minor

cols_to_remove <- match(c("Undergraduate.University...Cleaned",
                          "Major.1...Cleaned", "Major.2...Cleaned",
```

```
                    "Minor...Cleaned",
                    "Is.Math..Sci..or.Eng.Major.Minor"),
                  names(pa_academic))
cols_to_remove <- cols_to_remove[!is.na(cols_to_remove)]
pa_academic <- pa_academic[, -cols_to_remove]

#converting variables to dummy column & factors
pa_academic$Completed.Admissions.Process <-
as.factor(pa_academic$Completed.Admissions.Process)

pa_academic <- dummy_cols(pa_academic,  select_columns = c("stem",
"School.Selectivity", "Region.Preference.Level"),
                    remove_first_dummy = FALSE,
                    remove_selected_columns = TRUE)

str(pa_academic)



#academic partition

academic.idx <- createDataPartition(pa_academic$Completed.Admissions.Process, p=0.8,
list = FALSE )

train.academic.df <-pa_academic[academic.idx,]
test.academic.df <-pa_academic[-academic.idx,]


#handling class imbalance
train.academic.df <- ovun.sample(
  Completed.Admissions.Process ~ .,
  data   = train.academic.df,
  method = "both",   # hybrid oversampling + undersampling
  p      = 0.5,      # target 50/50 balance
  seed   = 1946
)$data


train.academic.df$Completed.Admissions.Process <-
as.factor(train.academic.df$Completed.Admissions.Process)
test.academic.df$Completed.Admissions.Process  <-
as.factor(test.academic.df$Completed.Admissions.Process)



# Check class proportions in each subset
prop.table(table(pa_academic$Completed.Admissions.Process))
prop.table(table(train.academic.df$Completed.Admissions.Process))
```

```
#MODEL 1: K-NEAREST NEIGHBORS (KNN)
model_knn <- train (Completed.Admissions.Process ~ .,
              data       = train.academic.df,
              method     = "knn",
              tuneLength = 10,                # tests k = 1 to 10
              preProcess = c("center", "scale"),
              trControl  = trainControl(method = "cv", number = 10)
             )

model_knn$bestTune
confusionMatrix(predict(model_knn, test.academic.df),
           test.academic.df$Completed.Admissions.Process,
           positive = "1")
```

```
> model_knn$bestTune
  k
1 5
> confusionMatrix(predict(model_knn, test.academic.df),
+                 test.academic.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1623 5514
         1 1259 6571

               Accuracy : 0.5475
                 95% CI : (0.5395, 0.5555)
    No Information Rate : 0.8074
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0684

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5437
            Specificity : 0.5632
         Pos Pred Value : 0.8392
         Neg Pred Value : 0.2274
             Prevalence : 0.8074
         Detection Rate : 0.4390
   Detection Prevalence : 0.5232
      Balanced Accuracy : 0.5534

       'Positive' Class : 1
```

```
#MODEL 2: NAIVE BAYES (NB)
modelLookup('naive_bayes')

tune_grid_nb <- expand.grid(
  laplace   = c(0,0.5, 1, 1.5, 2),
  usekernel = c(TRUE, FALSE),
  adjust    = c(0.75, 1, 1.25, 1.5,1.75,2)
  )

model_nb <- train(Completed.Admissions.Process ~ .,
                  data       = train.academic.df,
                  method     = "naive_bayes",
                  tuneGrid   = tune_grid_nb,
                  preProcess = c("center", "scale"),
                  trControl  = trainControl(method = "cv", number = 10))

model_nb$bestTune
confusionMatrix(predict(model_nb, test.academic.df),
            test.academic.df$Completed.Admissions.Process,
            positive = "1")
```

```
> model_nb$bestTune
  laplace usekernel adjust
7       0      TRUE   0.75
> confusionMatrix(predict(model_nb, test.academic.df),
+                 test.academic.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1914 5595
         1  968 6490

               Accuracy : 0.5615
                 95% CI : (0.5535, 0.5695)
    No Information Rate : 0.8074
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1248

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5370
            Specificity : 0.6641
         Pos Pred Value : 0.8702
         Neg Pred Value : 0.2549
             Prevalence : 0.8074
         Detection Rate : 0.4336
   Detection Prevalence : 0.4983
      Balanced Accuracy : 0.6006

       'Positive' Class : 1
```

```
#MODEL 3: DECISION TRESS (DT)
```

```
modelLookup('C5.0')

tune_grid_dt <- expand.grid(
  model  = c("rules", "tree"),
  winnow = TRUE,
  trials = seq(1, 9, 1)
)

model_dt <- train(Completed.Admissions.Process ~ .,
            data = train.academic.df,
            method = "C5.0",
            trControl = trainControl(method = "cv", number = 10),
            preProcess = c("center", "scale"),
            tuneGrid = tune_grid_dt)

model_dt$bestTune
confusionMatrix(predict(model_dt, test.academic.df),
          test.academic.df$Completed.Admissions.Process,
          positive = "1")
```

```
> model_dt$bestTune
   trials model winnow
10      1  tree   TRUE
> confusionMatrix(predict(model_dt, test.academic.df),
+                 test.academic.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1843 5605
         1 1039 6480

               Accuracy : 0.5561
                 95% CI : (0.5481, 0.5641)
    No Information Rate : 0.8074
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1096

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5362
            Specificity : 0.6395
         Pos Pred Value : 0.8618
         Neg Pred Value : 0.2474
             Prevalence : 0.8074
         Detection Rate : 0.4330
   Detection Prevalence : 0.5024
      Balanced Accuracy : 0.5878

       'Positive' Class : 1
```

```
#MODEL 4: ARTIFICIAL NEURAL NETWORKS (ANN)
modelLookup("nnet")
```

```
tune_grid_ann <- expand.grid(
          size = seq(5,15,5),
          decay = seq(0.1,0.7, 0.2))

model_ann <- train(Completed.Admissions.Process ~ .,
               data = train.academic.df,
               method = "nnet",
               metric = "Kappa",
               maxit = 50,
               preProcess = c("center", "scale"),
               trControl = trainControl(method = "cv", number = 5),
               tuneGrid = tune_grid_ann,
               trace = FALSE)

model_ann$bestTune
confusionMatrix(predict(model_ann, test.academic.df),
            test.academic.df$Completed.Admissions.Process,
            positive = "1")
```

```
> model_ann$bestTune
  size decay
9   15   0.1
> confusionMatrix(predict(model_ann, test.academic.df),
+                 test.academic.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1937 5667
         1  945 6418

               Accuracy : 0.5582
                 95% CI : (0.5502, 0.5662)
    No Information Rate : 0.8074
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1251

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5311
            Specificity : 0.6721
         Pos Pred Value : 0.8717
         Neg Pred Value : 0.2547
             Prevalence : 0.8074
         Detection Rate : 0.4288
   Detection Prevalence : 0.4919
      Balanced Accuracy : 0.6016

       'Positive' Class : 1
```

```
#MODEL 5: SUPPORT VECTOR MACHINE (SVM)
modelLookup('svmRadial')
```

```
tune_grid_svm <- expand.grid(
  sigma = c(0.01),
  C     = c(1, 2)
)

model_svm <- train(Completed.Admissions.Process ~ . ,
                   data = train.academic.df,
                   method = "svmRadial",
                   tuneGrid = tune_grid_svm,
                   preProcess = c("center", "scale"),
                   trControl = trainControl(method = "cv", number = 3))

model_svm$bestTune
confusionMatrix(predict(model_svm, test.academic.df),
                test.academic.df$Completed.Admissions.Process,
                positive = "1")
```

```
> model_svm$bestTune
  sigma C
2  0.01 2
> confusionMatrix(predict(model_svm, test.academic.df),
+                 test.academic.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1879 5231
         1 1003 6854

               Accuracy : 0.5835
                 95% CI : (0.5755, 0.5914)
    No Information Rate : 0.8074
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1406

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5671
            Specificity : 0.6520
         Pos Pred Value : 0.8723
         Neg Pred Value : 0.2643
             Prevalence : 0.8074
         Detection Rate : 0.4579
   Detection Prevalence : 0.5250
      Balanced Accuracy : 0.6096

       'Positive' Class : 1
```

Combined Approach

```
# Load libraries
library(class)
library(e1071)
library(nnet)
library(rpart)
library(caret)
library(dplyr)
library(ROSE)
library(pROC)
library(fastDummies)
library(NeuralNetTools)
library(tidyr)

set.seed(1946)


# STEP 1: LOAD DATA

pa.org <- read.csv("People Analytics at Teach For America Data Set.csv")
str(pa.org)
summary(pa.org)
head(pa.org)
sum(is.na(pa.org))


# Remove unnecessary columns
pa.df <- pa.org[, c(-1, -2, -5, -6, -7, -8, -(18:27), -(35:37), -39)]
head(pa.df)
str(pa.df)
summary(pa.df)



# STEP 2: CREATE COMBINED DATASET

# Behavioral + Academic variables

combined.df <- pa.df  # start from cleaned base dataset

# Behavioral Derivations
combined.df$Total.Essay.Length <- combined.df$Essay.1.Length +
                                  combined.df$Essay.2.Length +
                                  combined.df$Essay.3.Length
```

```
# Convert dates
combined.df$Sign.up.Date        <- as.Date(combined.df$Sign.up.Date,        format = "%d-
%m-%Y")
combined.df$Started.Date        <- as.Date(combined.df$Started.Date,        format = "%d-
%m-%Y")
combined.df$Submitted.Date      <- as.Date(combined.df$Submitted.Date,      format = "%d-
%m-%Y")
combined.df$Application.Deadline <- as.Date(combined.df$Application.Deadline, format =
"%d-%m-%Y")



# Behavioral engineered variables
combined.df$Days.to.Start   <- as.numeric(combined.df$Started.Date   -
combined.df$Sign.up.Date)
combined.df$Days.to.Submit  <- as.numeric(combined.df$Submitted.Date -
combined.df$Started.Date)
combined.df$Deadline.Gap    <- as.numeric(combined.df$Application.Deadline -
combined.df$Submitted.Date)



# Remove raw date columns
combined.df <- combined.df[, -
match(c("Sign.up.Date","Started.Date","Submitted.Date","Application.Deadline"),
                              names(combined.df))]



# Academic Derivations
combined.df$major_count <- apply(
  combined.df[, c("Major.1...Cleaned", "Major.2...Cleaned", "Minor...Cleaned")],
  1,
  function(x) sum(x != "none" & x != "" & !is.na(x))
)



# STEM binary variable
combined.df$stem <- combined.df$Is.Math..Sci..or.Eng.Major.Minor



# Remove categorical academic fields not needed after deriving major_count
cols_remove_academic <- match(
  c("Undergraduate.University...Cleaned",
    "Major.1...Cleaned","Major.2...Cleaned",
    "Minor...Cleaned",
    "Is.Math..Sci..or.Eng.Major.Minor"),
  names(combined.df)
```

```
)

cols_remove_academic <- cols_remove_academic[!is.na(cols_remove_academic)]
combined.df <- combined.df[, -cols_remove_academic]


# Remove negative time values

neg_cols <- c("Days.to.Start", "Days.to.Submit", "Deadline.Gap")
for (v in neg_cols) combined.df[[v]][combined.df[[v]] < 0] <- NA
combined.df <- tidyr::drop_na(combined.df, all_of(neg_cols))


# Convert outcome variable to factor
combined.df$Completed.Admissions.Process <-
as.factor(combined.df$Completed.Admissions.Process)


# Dummify academic categorical predictors
combined.df <- dummy_cols(combined.df,
                    select_columns = c("stem", "School.Selectivity",
"Region.Preference.Level"),
                    remove_first_dummy = FALSE,
                    remove_selected_columns = TRUE)

str(combined.df)



# STEP 3: TRAIN-TEST SPLIT
comb.idx <- createDataPartition(combined.df$Completed.Admissions.Process, p = 0.8, list =
FALSE)

train.combined.df <- combined.df[comb.idx, ]
test.combined.df  <- combined.df[-comb.idx, ]



# STEP 4: HANDLE CLASS IMBALANCE (ROSE)
train.combined.df <- ovun.sample(
  Completed.Admissions.Process ~ .,
  data = train.combined.df,
  method = "both",
  p = 0.5,
  seed = 1946
)$data

train.combined.df$Completed.Admissions.Process <-
as.factor(train.combined.df$Completed.Admissions.Process)
```

```
test.combined.df$Completed.Admissions.Process  <-
as.factor(test.combined.df$Completed.Admissions.Process)

# Check class proportions
prop.table(table(train.combined.df$Completed.Admissions.Process))
prop.table(table(test.combined.df$Completed.Admissions.Process))


# MODEL 1: KNN
model_knn <- train(Completed.Admissions.Process ~ .,
              data = train.combined.df,
              method = "knn",
              tuneLength = 10,
              preProcess = c("center", "scale"),
              trControl = trainControl(method = "cv", number = 10))

model_knn$bestTune
confusionMatrix(predict(model_knn, test.combined.df),
           test.combined.df$Completed.Admissions.Process,
           positive = "1")
```

```
> model_knn$bestTune
  k
1 5
> confusionMatrix(predict(model_knn, test.combined.df),
+                 test.combined.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1552 5358
         1 1183 6145

               Accuracy : 0.5406
                 95% CI : (0.5324, 0.5488)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0643

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.5342
            Specificity : 0.5675
         Pos Pred Value : 0.8386
         Neg Pred Value : 0.2246
             Prevalence : 0.8079
         Detection Rate : 0.4316
   Detection Prevalence : 0.5147
      Balanced Accuracy : 0.5508

       'Positive' Class : 1
```

```
# MODEL 2: NAIVE BAYES
tune_grid_nb <- expand.grid(
  laplace   = c(0,0.5,1,1.5,2),
  usekernel = c(TRUE, FALSE),
  adjust    = c(0.75,1,1.25,1.5,1.75,2)
)

model_nb <- train(Completed.Admissions.Process ~ .,
                  data = train.combined.df,
                  method = "naive_bayes",
                  tuneGrid = tune_grid_nb,
                  preProcess = c("center","scale"),
                  trControl = trainControl(method = "cv", number = 10))

model_nb$bestTune
confusionMatrix(predict(model_nb, test.combined.df),
                test.combined.df$Completed.Admissions.Process,
                positive = "1")
```

```
> model_nb$bestTune
  laplace usekernel adjust
7       0      TRUE   0.75
> confusionMatrix(predict(model_nb, test.combined.df),
+                 test.combined.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 2234 6750
         1  501 4753

               Accuracy : 0.4907
                 95% CI : (0.4825, 0.499)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.123

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.4132
            Specificity : 0.8168
         Pos Pred Value : 0.9046
         Neg Pred Value : 0.2487
             Prevalence : 0.8079
         Detection Rate : 0.3338
   Detection Prevalence : 0.3690
      Balanced Accuracy : 0.6150

       'Positive' Class : 1
```

```
# MODEL 3: DECISION TREE
```

```
tune_grid_dt <- expand.grid(
  model  = c("rules","tree"),
  winnow = TRUE,
  trials = seq(1, 9, 1)
)

model_dt <- train(Completed.Admissions.Process ~ .,
              data = train.combined.df,
              method = "C5.0",
              tuneGrid = tune_grid_dt,
              preProcess = c("center","scale"),
              trControl = trainControl(method = "cv", number = 10))

model_dt$bestTune
confusionMatrix(predict(model_dt, test.combined.df),
            test.combined.df$Completed.Admissions.Process,
            positive = "1")
```

```
> model_dt$bestTune
   trials model winnow
17      8  tree   TRUE
> confusionMatrix(predict(model_dt, test.combined.df),
+                 test.combined.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1137 2978
         1 1598 8525

               Accuracy : 0.6786
                 95% CI : (0.6709, 0.6863)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1315

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.7411
            Specificity : 0.4157
         Pos Pred Value : 0.8421
         Neg Pred Value : 0.2763
             Prevalence : 0.8079
         Detection Rate : 0.5987
   Detection Prevalence : 0.7110
      Balanced Accuracy : 0.5784

       'Positive' Class : 1
```

MODEL 4: ARTIFICIAL NEURAL NETWORK

```
tune_grid_ann <- expand.grid(
  size  = seq(5,15,5),
  decay = seq(0.1,0.7,0.2)
)

model_ann <- train(Completed.Admissions.Process ~ .,
                   data = train.combined.df,
                   method = "nnet",
                   tuneGrid = tune_grid_ann,
                   metric = "Kappa",
                   maxit = 50,
                   preProcess = c("center","scale"),
                   trControl = trainControl(method = "cv", number = 5),
                   trace = FALSE)

model_ann$bestTune
confusionMatrix(predict(model_ann, test.combined.df),
                test.combined.df$Completed.Admissions.Process,
                positive = "1")
```

```
> model_ann$bestTune
   size decay
10   15   0.3
> confusionMatrix(predict(model_ann, test.combined.df),
+                 test.combined.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1726 4350
         1 1009 7153

               Accuracy : 0.6236
                 95% CI : (0.6156, 0.6316)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1726

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.6218
            Specificity : 0.6311
         Pos Pred Value : 0.8764
         Neg Pred Value : 0.2841
             Prevalence : 0.8079
         Detection Rate : 0.5024
   Detection Prevalence : 0.5733
      Balanced Accuracy : 0.6265

       'Positive' Class : 1
```

```
# MODEL 5: SVM
```

```
tune_grid_svm <- expand.grid(
  sigma = c(0.01),
  C     = c(1, 2)
)

model_svm <- train(Completed.Admissions.Process ~ .,
                   data = train.combined.df,
                   method = "svmRadial",
                   tuneGrid = tune_grid_svm,
                   preProcess = c("center","scale"),
                   trControl = trainControl(method = "cv", number = 3))

model_svm$bestTune
confusionMatrix(predict(model_svm, test.combined.df),
                test.combined.df$Completed.Admissions.Process,
                positive = "1")
```

```
> model_svm$bestTune
  sigma C
2  0.01 2
> confusionMatrix(predict(model_svm, test.combined.df),
+                 test.combined.df$Completed.Admissions.Process,
+                 positive = "1")
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1692 4272
         1 1043 7231

               Accuracy : 0.6267
                 95% CI : (0.6187, 0.6347)
    No Information Rate : 0.8079
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1705

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.6286
            Specificity : 0.6186
         Pos Pred Value : 0.8739
         Neg Pred Value : 0.2837
             Prevalence : 0.8079
         Detection Rate : 0.5079
   Detection Prevalence : 0.5811
      Balanced Accuracy : 0.6236

       'Positive' Class : 1
```